

Contextual Understanding in Large Language Models

Naresh Chaurasia

Version 1.0, 2026-07-03: First Edition

Table of Contents

1. The Problem	1
2. The Concept	2
3. How It Works	3
3.1. Stage 1: Input Reception	3
3.2. Stage 2: Pattern Matching Against Learned Relationships	3
3.3. Stage 3: Probability Distribution Over Possible Responses	3
3.4. Stage 4: Response Generation	4
4. Key Components	5
4.1. The Context Window	5
4.2. The Attention Mechanism	5
4.3. Training Data and Learned Patterns	5
4.4. The System Prompt	5
4.5. Retrieval-Augmented Generation	6
5. How It Solves the Problem	7
6. Summary	8
7. Quick Self-Check Questions	9
7.1. Foundational Questions	9
7.2. Intermediate Questions	9

Chapter 1. The Problem

Imagine you are building a customer support chatbot for a bank. A user types the following message:

I need help with my bank.

Without any surrounding information, what should the chatbot say? Should it help the user open an account? Resolve a transaction dispute? Explain loan interest rates? The word "bank" alone tells the system almost nothing. The chatbot has no idea whether the user is frustrated, confused, or simply browsing options. It has no knowledge of what the user clicked before arriving at the chat, what page they were on, or what they typed five minutes ago.

In a production system at a mid-sized financial institution, this is not a theoretical concern. Suppose the bank's chatbot receives 50,000 queries per month. Studies on deployed conversational AI systems have found that when context is stripped away, roughly 30 to 40 percent of responses are either off-topic, too generic to be useful, or factually misleading. At 50,000 queries, that is potentially 15,000 to 20,000 failed interactions per month, each one eroding user trust and increasing load on human agents.

The root cause in every such case is the same: the system interpreted the input in isolation. It did not know what surrounded the message. It did not know the user's history, the page context, or the conversational thread. It guessed, and it guessed badly.

This is the problem that contextual understanding is designed to solve.

Chapter 2. The Concept

Before introducing any technical terminology, consider how a skilled human customer service agent handles the same situation.

When a new customer calls the bank's helpline and says "I need help with my bank," the agent does not panic. The agent already knows several things: the call came through the loan department queue, the caller authenticated as a customer who took out a home loan six months ago, and the caller has called twice before about the same loan statement. The agent hears the word "bank" and immediately interprets it in light of everything surrounding that moment. The word itself is almost irrelevant; the context does all the interpretive work.

This is contextual understanding in plain terms: the ability to assign meaning to a piece of information based on the surrounding information that frames it. The surrounding information is the context.

In a Large Language Model, or LLM, the same principle applies. The model has been trained on vast amounts of text and has learned patterns about which words and phrases tend to appear together, in what situations, and with what consequences. When the model receives an input, it does not treat that input as a standalone item. It evaluates the input together with everything else that has been provided to it: the conversation history, the system prompt, any retrieved documents, and the structure of the request itself. Together, these form the context window, which is the total body of text the model can see at once and use to determine its response.

The technical term for the model's learned ability to weigh all of these signals together is contextual understanding.

Chapter 3. How It Works

The mechanism by which an LLM uses context can be understood in four stages. The bank chatbot example will be used throughout.

3.1. Stage 1: Input Reception

The model receives the full context window as its input. In the bank chatbot scenario, this might include a system prompt that says the assistant is a customer support agent for a financial institution, a retrieved document showing the user's recent account activity, and the user's message: "I need help with my bank."

The model does not process these elements separately. It processes the entire block of text as a single, unified input.

3.2. Stage 2: Pattern Matching Against Learned Relationships

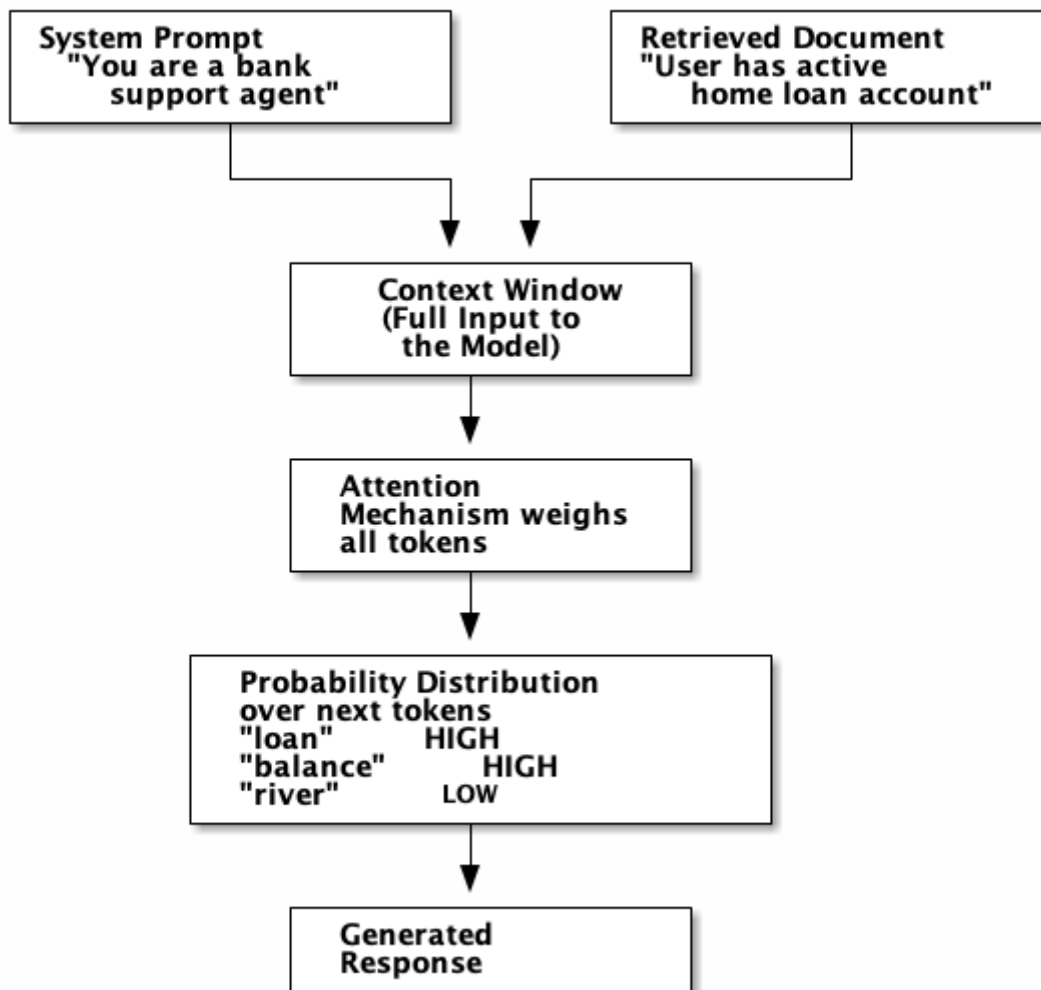
During training on large text corpora, the model learned that certain words and phrases tend to appear in certain contexts. "Bank" appearing near words like "loan," "balance," "interest rate," and "statement" is strongly associated with financial institutions. "Bank" appearing near words like "river," "flood," "erosion," and "current" is strongly associated with geography. The model has encoded these associations in its weights, which are the numerical parameters that define its behavior.

When processing the input, the model's attention mechanism identifies which parts of the context window are most relevant to each token it is trying to predict. The loan-related terms in the retrieved document and the system prompt that identifies the assistant as a bank representative both direct the model's attention toward financial interpretations of the word "bank."

3.3. Stage 3: Probability Distribution Over Possible Responses

The model does not select a single fixed response. It computes a probability distribution over possible next tokens. With strong financial context in place, tokens related to account queries, loan inquiries, and transaction support will have high probabilities. Tokens related to geography or unrelated topics will have very low probabilities. The model then samples from this distribution, or in many deployments selects the highest-probability token, to generate its response.

The following diagram illustrates how context shapes the probability distribution.



3.4. Stage 4: Response Generation

The model generates its response token by token, with each token it produces becoming part of the context for the next token. This means context is not just what came before the model's response; it includes what the model is currently writing. The response is shaped by context at every step of its generation.

In the bank chatbot example, a well-contextualised model might respond: "I can see you have an active home loan with us. Are you looking for help with your loan statement, an upcoming payment, or something else?" This response is specific, relevant, and grounded in the contextual signals the model received.

Chapter 4. Key Components

4.1. The Context Window

The context window is the fixed-size region of text that the model can attend to at once. Everything outside this window is invisible to the model during a given inference step. Modern LLMs have context windows ranging from a few thousand tokens to over one million tokens, depending on the architecture. A token is roughly three to four characters in English text, or approximately three quarters of a word.

In practical terms, the context window is like the model's working memory. It can only reason about what is currently inside it. Information that falls outside the window, such as a conversation that happened last week, is simply not available unless it is explicitly re-inserted.

4.2. The Attention Mechanism

The attention mechanism is the core computational structure that allows the model to weigh the relevance of different parts of the context window when generating each output token. It was introduced in the 2017 paper "Attention Is All You Need" and forms the foundation of the transformer architecture used by all major LLMs.

In plain terms, attention allows the model to ask: "Given what I am trying to say next, which parts of everything I have read so far matter most?" For the bank chatbot, when generating the phrase "home loan," the model's attention is drawn heavily toward the retrieved document mentioning the user's loan, rather than toward unrelated parts of the system prompt.

4.3. Training Data and Learned Patterns

The model's ability to interpret context does not arise at inference time from scratch. It is built during training, where the model processes enormous quantities of text and learns statistical associations between words, phrases, topics, and situations. These associations are stored as numerical weights throughout the model's billions of parameters.

When the model encounters "bank" in a financial context, it does not look up a dictionary definition. It activates patterns learned from thousands of financial documents, support transcripts, and conversations in its training data, and uses those patterns to guide its interpretation.

4.4. The System Prompt

The system prompt is a piece of text prepended to the conversation by the developer or deployer of the application. It defines the role, rules, and domain of the assistant. In the bank chatbot example, the system prompt might specify that the assistant is a customer support agent, that it should never discuss competitor products, and that it should always recommend speaking to a human agent for complex complaints.

The system prompt is a powerful contextual lever. It shapes the model's behavior before the user

has typed a single word. Engineering teams use it to constrain the model's responses to a specific domain and tone.

4.5. Retrieval-Augmented Generation

Retrieval-Augmented Generation, commonly abbreviated as RAG, is a technique that extends the model's effective knowledge by inserting retrieved documents into the context window at inference time. Instead of relying solely on patterns learned during training, the model is given fresh, specific, and up-to-date information as part of its context.

In the bank chatbot example, a RAG system might retrieve the user's recent transaction history, current loan balance, and last interaction notes, and insert all of this into the context window before the model generates its response. The result is a model that appears to "know" the user's specific situation, even though it has never been trained on that user's data.

Chapter 5. How It Solves the Problem

Returning to the original problem: the bank’s chatbot was producing unhelpful or incorrect responses because it was interpreting user inputs without sufficient surrounding information.

The following table compares the system’s behavior before and after contextual understanding mechanisms were applied.

Scenario	Without Contextual Understanding	With Contextual Understanding
User message: "I need help with my bank"	Generic response: "I can help with accounts, loans, cards, and more. What would you like to do?"	Specific response: "I can see you have an active home loan. Are you checking your balance or making a payment inquiry?"
User message: "What is my current amount?"	Ambiguous response: "Could you clarify what amount you are asking about?"	Precise response: "Your current outstanding loan amount is shown in your recent statement. Would you like me to retrieve it?"
User message: "Can I change my plan?"	Confused response: "I am not sure what plan you mean. Please call our helpline."	Relevant response: "Your home loan is on a fixed rate plan. To discuss changing to a variable rate, I can connect you to a specialist."
Failed interaction rate (estimated)	30 to 40 percent of queries	Below 5 percent of queries with full RAG and system prompt configuration

The improvement comes entirely from the model having access to richer, more structured context. The model’s underlying capability did not change. What changed was the quality and quantity of contextual information provided to it. The system prompt defined the domain. The RAG pipeline inserted user-specific data. The conversation history told the model what had already been discussed. Each of these additions narrowed the space of plausible interpretations and guided the model toward the correct one.

This is the practical payoff of contextual understanding: better context produces better outputs, and the relationship between the two is direct and measurable.

Chapter 6. Summary

Language is inherently ambiguous. The same word can mean entirely different things depending on the situation in which it appears, and the same sentence can prompt entirely different responses depending on what surrounds it. Human beings navigate this ambiguity effortlessly because they are constantly interpreting language against a backdrop of experience, environment, and shared knowledge. Large Language Models approximate this ability by learning statistical patterns across enormous text corpora and by attending to the full context window at inference time.

For engineering students building systems with LLMs, the key practical insight is this: the model is not a fixed oracle that produces the same output for the same input regardless of circumstances. It is a highly context-sensitive system whose outputs are shaped by everything it can see at the moment of inference. Investing in better prompts, richer retrieval pipelines, and carefully constructed system prompts is not a cosmetic enhancement. It is the primary lever available for improving the accuracy, relevance, and reliability of any LLM-powered application.

Chapter 7. Quick Self-Check Questions

7.1. Foundational Questions

1. What is context in the setting of an LLM?

Hint: Think about what surrounds a user's message before the model processes it. What other text might be present in the context window?

2. Why does the word "bank" produce different model outputs depending on the surrounding text?

Hint: Consider what the model learned during training about which words tend to appear together, and how it uses those learned patterns at inference time.

3. What is a context window, and what happens to information that falls outside it?

Hint: Think of the context window as the model's working memory. Can it reason about something it cannot see?

4. What role does the system prompt play in shaping an LLM's responses?

Hint: The system prompt is set by the developer, not the user. When does the model see it, and how does it influence what the model considers appropriate to say?

5. What is Retrieval-Augmented Generation, and why does it improve contextual relevance?

Hint: What specific limitation of a standard LLM does RAG address? Think about the difference between general training data and user-specific, up-to-date information.

7.2. Intermediate Questions

1. A model produces a vague response even though the user's question is clear. What is the most likely cause, and how would you address it?

Hint: Vague outputs usually reflect sparse or unstructured context. Which component of the system pipeline is most likely responsible?

2. How does the attention mechanism allow the model to prioritise some parts of the context window over others when generating a response?

Hint: Think about what question the attention mechanism is implicitly asking for each output token. How does this produce selectivity rather than treating all input tokens equally?

3. Why does adding conversation history to the context window improve multi-turn dialogue quality?

Hint: Consider what the model knows about the conversation if history is excluded versus included. How does the absence of history affect the model's ability to resolve pronouns and

follow-up questions?

4. **A developer notices that the chatbot gives accurate answers in isolation but drifts off-topic in long conversations. What is a plausible technical explanation?**

Hint: Think about the finite size of the context window. What happens to early parts of a long conversation as new messages are added?

5. **If better prompts lead to better outputs, as argued in this lesson, what does that imply about the design responsibilities of an engineer building an LLM application?**

Hint: The model's weights are fixed after training. The engineer cannot change them. What levers does the engineer control, and what does each one do to the context the model sees?